

Elevator Pitch

1. I built an end-to-end AI-powered Medical Chatbot that answers health-related queries using The Gale Encyclopedia of Medicine (700+ pages).
2. It extracts medical text, splits it into semantic chunks, generates vector embeddings, and stores them in Pinecone
3. Using LangChain and a lightweight LLM (FLAN-T5) integrated with Flask web UI.
4. I implemented a full Retrieval-Augmented Generation (RAG) pipeline that performs intelligent search and provides concise, source-grounded medical answers
5. I handled the entire pipeline—from data extraction, chunking, and embedding to vector database design, backend integration, and UI deployment.

Step by Step

1. **Project Goal :** Build a reliable medical QA chatbot that answers questions using a large offline medical encyclopaedia (Gale encyclopaedia of Medicine), avoiding hallucinations by grounding answers in the book
2. **Data Collection**
 1. I used “The Gale Encyclopedia of Medicine” — a large 700+ page medical encyclopedia PDF as the knowledge source.
 2. **How :** It was loaded using LangChain’s PyPDFLoader which extracts text page by page.
 3. **Why :** Because the chatbot needs a real knowledge base to answer accurately instead of hallucinating.
3. **Text Preprocessing and Chunking**
 1. **What :** Split long text into overlapping chunks (chunks_size=500, overlap=20).
 2. **How :** RecursiveCharacterTextSplitter - prevents chopping sentences and ensures context overlap for retrieval
 3. **Why :** LLMs have limited input context length ; Smaller chunks = better granularity for retrieval ; Overlap ensures continuity between chunks.
 4. **Concept :** Chunking balances retrieval granularity (specificity) and context size (LLM’s understanding scope).
4. **Embedding Generation**
 1. **What :** Convert each chunk to a fixed vector (embedding_dim=384)
 2. **How :** I used HuggingFace Embeddings (all-MiniLM-L6-v2) model:
 1. embeddings = HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2')
 3. **Why :** Embeddings convert each text chunk into a vector representation (384 dimensions) capturing semantic meaning — similar chunks have similar vectors.
 4. **Concept :** An embedding model converts text into numerical form (vector) so the computer can compare meanings via cosine similarity.

5. Vector Database - Pinecone

1. **What** : I created a Pinecone index to store these embeddings:
2. **How** : `pc.create_index(name="medicalbot", dimension=384, metric="cosine")`
3. **Why** : Local vector storage isn't feasible for huge datasets (memory issue) ; Pinecone is cloud-hosted, scalable, and allows semantic search in milliseconds.
4. **Concept** : Vector Databases store and retrieve similar vectors efficiently ; Instead of exact keyword match, they find meaning-based similarity using cosine distance.

6. Storing & Retrieving Vectors

1. Stored all embedded chunks into Pinecone using:
`PineconeVectorStore.from_documents(documents=text_chunks, index_name=index_name, embedding=embeddings)`
2. Then loaded it later via:
`PineconeVectorStore.from_existing_index(index_name=index_name, embedding=embeddings)`
3. This allows reusing the index without recreating it every time

7. Retrieval-Augmented Generation (RAG)

1. This Is the core AI architecture of the project.
2. Working :
 1. User asks a question -> "what is acne?"
 2. Question is converted to embedding vector
 3. Vector is compared in Pinecone -> retrieves top 3 similar chunks
 4. These chunks + user question are passed to OpenAI LLM
 5. LLM generates an answer based only on retrieved context
3. Why : it combines retrieval (knowledge grounding) + generation (language fluency) = factual + natural responses.
4. Concept : RAG = Retrieval + Generation pipeline ; solves hallucination problem by giving LLM real context

8. LLM (OpenAI Model)

1. Used OpenAI LLM with parameters : `llm = OpenAI(temperature=0.4, max_tokens=500)`
2. Why : Already hosted on cloud (no local GPU needed) ; very accurate on language generation tasks
3. Concepts : temperature: controls creativity (0 = factual, 1 = creative) ; max_tokens: limits response length

9. Prompt Engineering

1. **System Prompt**: "You are an assistant for question-answering tasks. Use retrieved context to answer concisely in 3 sentences."
2. Why : to make LLM responses consistent , context - aware , ad short.
3. Concept : Prompt engineering shapes how LLM behaves- acts as instruction tuning during inference.

10. Flask Web App (Frontend)

1. Built a simple Flask app (app.py) + HTML/CSS chat UI.
2. The UI (chat.html + style.css) mimics a real chatbot interface.

3. User enters a query -> flask sends to / get route -> backend generates reply via RAG -> returns to frontend dynamically
4. Why : lightweight , prefect for small web apps and REST endpoints

11. Deployment & Environment

1. Used .env to securely store API keys (OpenAI , Pinecone).
2. Used requirements.txt for dependency management,
3. Created virtual environment medibot for isolation
4. Can be hosted easily on AWS / Render /HuggingFace Spaces

Technologies used

1. LangChain

1. It is a powerful open-source framework that makes it easy to build complex AI applications using Large Language Models (LLMs) like GPT or FLAN-t5
2. In simple terms - LangChain helps you connect an LLM with external data sources (such as PDFs, databases, or APIs) and control how the model uses that data to produce meaningful and accurate responses
3. It handled : Text Chunking ; embedding & retrieval ; response generation pipeline

2. Pine Cone

1. Vector Database
2. Stores embeddings and retrieves most similar chunks using cosine similarity
3. **Why** : Local vector storage isn't feasible for huge datasets (memory issue) ; Pinecone is cloud-hosted, scalable, and allows semantic search in milliseconds.
4. **Concept** : Vector Databases store and retrieve similar vectors efficiently ; Instead of exact keyword match, they find meaning-based similarity using cosine distance.

3. HuggingFace Embeddings

1. NLP Model
2. Converts text chunks into numerical vectors for semantic understanding

4. Open AI GPT Model

1. LLM
 1. Large language Model - an advanced type of artificial intelligence trained on massive amounts of text data to understand and generate Human-like language.
 2. In simple words - an LLM can read , understand and write text just like a person
 3. Eg : GPT , FLAN - T5 , BERT , LLaMA , gemini , etc
 4. These can perform : answer questions , summarising documents , translating languages , generating code or content
2. Generates natural language answers using retrieved medical text

5. Flask

1. It is a lightweight python web framework used to build web application and APIs quick and easily .
2. It's called a micro-framework because it gives us the essentials - routing, request handling, and templates - without forcing u into a heavy structure
3. Can think it as the bridge between Ai model/ backend logic and the user interface (frontend)

4. App initialisation -> Routing (URL endpoints) -> Handling user input -> connecting backend logic (LangChain + Pinecone + LLM) -> Running Server
5. In my project : displays the chat UI to user -> takes medical queries as input -> sends them to backend(LangChain + Pinecone + LLM) -> receives and shows the AI-generated answer in the browser

6. HTML + CSS + JS

1. HTML(HyperText Markup Language) : is it like the skeleton of a webpage , it defines the structure and content - headings , buttons , forms , paragraphs, etc
2. CSS (cascading style sheets) : it adds style and layout to HTML elements - colours , fonts , spacing , alignment , animations , etc
3. JavaScript (JS) : It adds life and logic to webpage - handling user interactions, sending API requests , updating data dynamically

7. .env : Security , keeps API keys secret and separate from source code